Module 0: Introduction

Course Overview

COP4020: Programming Language Concepts

Welcome!

I’m your Instructor,

Blake Anderson.

My Background

MERCURY

UF Herbert Wertheim College of Engineering
Department of Computer & Information Science & Engineering
UNIVERSITY *of* FLORIDA

Caribou

infotech

Rhovas

SPONGE POWERED

1.

About `this` Course

`assert this instanceof COP4020;`

COP4020 (PLC) At A Glance

1: Language Implementation

Objective: Learn how programming languages operate at a lower-level to build up to higher-level abstractions.

Lectures, Quizzes, & Project:

- Interpreter: Lexer/Parser/Evaluator
- Compiler: Analyzer/Generator

2: Language Design/Features

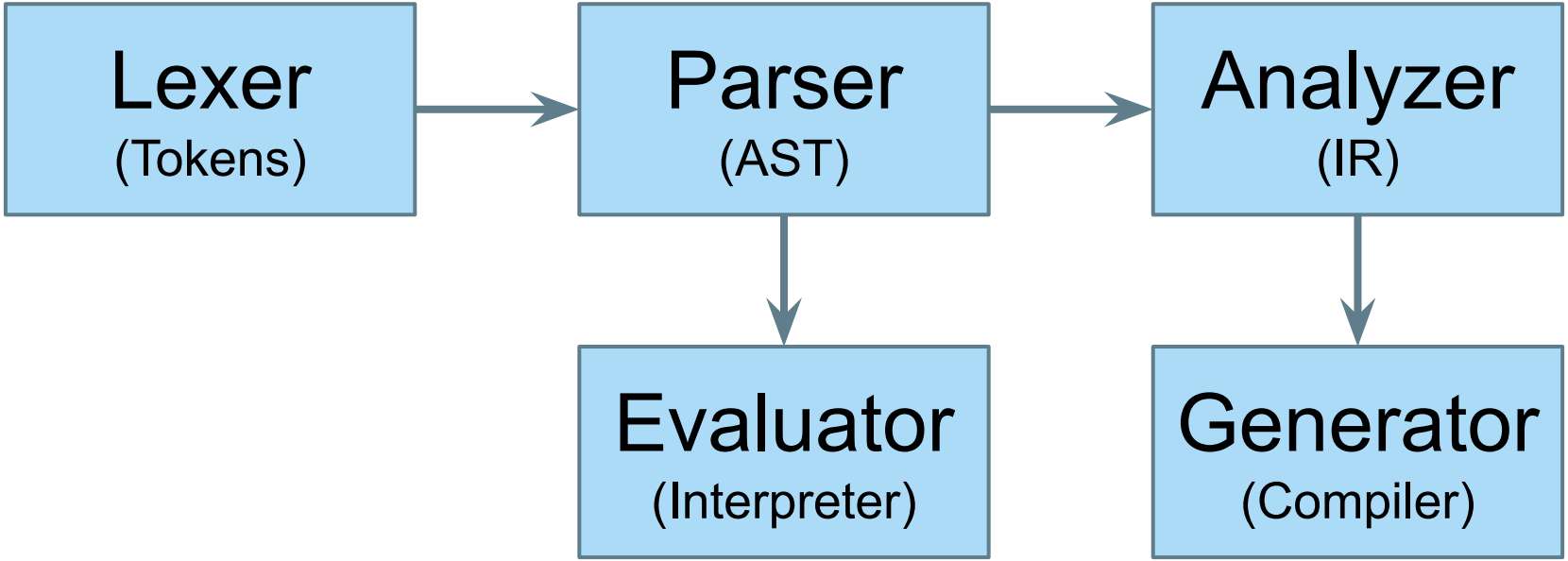
Objective: Learn different approaches for solving programming problems used by a wide variety of languages and their tradeoffs.

“Tangent” Lectures & Excercises:

- Error Handling, Syntax Sugar, Functional Programming, Types, Macros & DSLs.

This is where the fun begins!

The Big Idea (!)


Lexer
(Tokens)
Parser
(AST)
Analyzer
(IR)
Evaluator
(Interpreter)
Generator
(Compiler)

“Don't reinvent the wheel, unless you plan on learning more about wheels”

~ Jeff Atwood (StackOverflow)

A Few Disclaimers

1. Second Time Teaching PLC

Have a pretty strong grasp on things now, but still room for improvement.

- Helped design the current version of the course with Pete Dobbins.
  - And taught it last semester.
- My background is programming languages (thesis, Rhovas, & OOP).
- Many TA's from last semester.

2. Now With (More) Design!

Previous iterations have focused on the project (impl) and not language design.

- Tangent lectures & Exercises will introduce design concepts and allow experimenting.
  - Bonus for "weird" langs like Zig & Elixir.
- Building lectures & assignments for this is tricky as there's a wide variety of concepts.
  - Expecting this to bit freeform/wonky.

Please share insight/feedback! When (not if) issues arise, we'll address them *fairly.*

2.

Syllabus Review

Does anyone actually read these things?

Assignments

Checkpoints: 33%

Implement a real, working language from scratch.

Interpreter:

- Lexer, 1 test: 5%
- Parser, 2 tests: 7.5%
- Evaluator, 2 tests: 8%

Compiler:

- Analyzer, 2 tests: 7.5%
- Generator, 1 test: 5%

Finale: 33%

- Resubmit checkpoints.
- Effectively, recover up to 50% of lost points.

Test Submissions: 4%

Early submissions for preliminary feedback.

- 7 test 1's, 2 drops: 0.4%
- 5 test 2/3's: 0.4%

Quizzes: 15%

Reinforce lecture content and key concepts.

- 6 total, 1 drop: 3%

Exercises (NEW*): 15%

Explore various languages, features, & tradeoffs.

- Regex, 1 test: 3%
- 5 more, 1 drop: 3%

Last Semester's Advice

Instructor Advice

Ask questions, even if they seem basic. Use office hours to your advantage.

- Questions on course material from lectures, quizzes, resources, etc.
- Questions on assignments, <u>especially with specifications.</u>
- Or whatever else is helpful
  - Java, language features, etc.

Student Advice

What advice would you give to students taking this class next semester?

1. Start projects early to allow time for questions and utilize test submissions.
2. Attend (and participate in) lectures. There's more going on than just the slides!
3. Review key concepts, especially around terminology. Quizzes require understanding!

Experimenting with different languages and comparing concepts helps makes it "click".

3.

PLC Motivation

Programming languages are our friends!

1

You will use more than one language!

- Most tech stacks have multiple languages.
  - frontend, backend, scripting, configuration...
- Languages are always being updated/replaced.

```jsx
// component.jsx

function Hello({name}) {
  return (
    <h1>Hello, {name}!</h1>
  );
}
```

```js
// component.js

function Hello({name}) {
  return React.createElement(
    "h1",
    {},
    "Hello, ",
    name,
    "!",
  );
}
```

Question: What language is React?

2

Understanding the core concepts makes it easier to learn languages & design software.

- Similar languages solve similar problems similarly.
  - Still, differences can completely change design!
- Languages ~~steal~~ adapt ideas from others too!

```erlang
% server.erl

loop() -> receive
  {Sender, Num} ->
    Sender ! Num,
    loop()
  end.

Server = spawn(fun loop/0).
send(Server, {self(), 1})
```

```swift
// Server.swift

actor Server {
  func echo(num: Int) -> Int {
    return num;
  }
}

var server = Server()
await server.echo(1)
```

From 1980s Telecoms to 2020s Telephones

3

Knowledge of languages helps understand runtime behavior (especially performance).

- Languages define execution behavior - if you don't know how it's run, how do you know it works?
- Frameworks in particularly (e.g. React, Rails) often involve "clever" operations with a lot of nuance.

4

They're just really cool!

- Languages are the "One True Abstraction".
- New developments in languages & tools.

4.

Next Week in PLC

What's on deque?

Next Week's Schedule (Drop/Add)

Mon 8/25: Design by Contract

Overview of computational thinking, specifications, and testing theory.

- Introduction to Software Design.
- Preconditions & postconditions.
- Test coverage, equivalence classes, and boundary/edge cases.

Wed 8/27: Regex (& Testing)

Introduction to regular expressions; practical application of testing.

- Regex and languages 101.
- Application of testing fundamentals.
- Preview class expectations, project structure, and Java/IntelliJ/JUnit.

Exercise 1: Regex Testing

Fri 8/29 - Wed 9/3: Java 101

Module 0: Introduction

Course Overview

COP4020: Programming Language Concepts

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals